stabilizer-python: narrow structure report

This note summarizes the **layout** of `stabilizer-python/` and focuses on **phase / sign tracking** in the stabilizer tableau. It is intentionally short.

Package layout

Path	Role
`stabilizer_python/tableau.py`	Core simulator: **Aaronson–Gottesman-style tableau** (`StabilizerState`), gates, **Z measurement**, row ops, **phase bits**, and **CHP-style debug output** (`format_chp_printstate`, `format_xz_binary_matrices`, `format_tableau_debug`).
`stabilizer_python/circuit.py`	**Builder**: frozen `Op` records; `Circuit` with `h`, `s`, `x`, `z`, `cnot`, `mz(q, key=…)`; `extend`; `run(state)` returns **measurement outcome bits**.
`stabilizer_python/linear_algebra.py`	**GF(2)** helpers: `gaussian_elimination_gf2`, `rank_gf2` (used by tests / tooling, not inside every gate step).
`stabilizer_python/codes.py`	**QEC helpers**: `BitFlip3Code`, `Shor9Code` (encoders, syndrome read/measure, X correction); `run_2qubit_bell`, `bitflip3_encode_zero_state`.
`stabilizer_python/examples/`	Runnable demos: Bell (`two_qubit_bell`), bit-flip (`bitflip3_demo`), Shor (`shor9_demo`).
`tests/`	Tableau / Bell parity (`test_two_qubit_bell`), bit-flip syndrome + correction (`test_bitflip3`), GF(2) (`test_gaussian_elimination`).
`x-gate-proof.md`	Standalone write-up: why **X conjugation** flips row sign when `z_q = 1` (Z and Y on that qubit).
`references/`	Gottesman–Knill reference notebooks (not imported by the package).

Public API (`__init__.py`): `StabilizerState`, `Circuit`, `gaussian_elimination_gf2`, `rank_gf2`, and the `codes` submodule.

Phase and sign tracking (the important part)

The state is stored as a **2n × n** binary tableau: per-row Pauli on each qubit via `(x_mat[r][q], z_mat[r][q])` bits (X/Z decomposition). Rows `0..n-1` are **destabilizers**, rows `n..2n-1` are **stabilizers**.

`r_phase`: only global ± on each row

`StabilizerState` keeps `r_phase[r] ∈ {0,1}` interpreted as an overall **(−1)** sign on that row's Pauli (see docstring in `tableau.py`). The implementation does **not** store full **i**-phases on rows for general Pauli tracking; instead it uses a **mod-4 i-exponent only inside row multiplication**, then folds to **±** (`exp_i == 2` → flip sign). That is enough for the Clifford + Pauli measurement algebra used here.

`_row_mult_phase` (Pauli product phases)

When two single-qubit Pauli factors multiply, **order matters** (e.g. (XZ = iY) vs (ZX = -iY)). The helper `_row_mult_phase(x1,z1,x2,z2)` returns an **exponent of (i)** (mod 4) for that pair, with **explicit branches** for all non-commuting pairs (XZ/ZX, ZY/YZ, YX/XY) and `0` for identity or matching Paulis; invalid bit pairs raise `ValueError`. `_rowmult` sums:

- existing row signs as +2 per row if `r_phase` is set,
- plus `_row_mult_phase` over all qubits,

then XORs the X and Z bit vectors and sets the new row sign from `exp_i % 4`.

Gate-induced sign updates

- **`h(q)`**: swaps X/Z on column `q`; if the cell is **Y** (`x & z`), flips `r_phase[r]` (Y picks up a sign under H in this bookkeeping).
- **`s(q)`**: phase gate on Z leg (`z ^= x`); again **Y** flips `r_phase[r]`.
- **`x(q)`**: conjugation flips sign when the row has **Z or Y** on `q` (`z_mat[r][q] == 1`). See `x-gate-proof.md` for the case analysis.
- **`z(q)`**: conjugation flips sign when the row has **X or Y** on `q` (`x_mat[r][q] == 1`).
- **`cnot(c,t)`**: binary update of X/Z plus a **conditional sign** from the Aaronson–Gottesman rule (`if xc & zt & (xt ^ zc ^ 1): r_phase[r] ^= 1`).

Measurement (`measure_z`)

- **Random** branch: finds a stabilizer row with X on `q`, uses `_rowmult` to clear other Xs, swaps rows, rewrites tableau so the measured generator is (Z_q) with `r_phase[n+q] = outcome`.
- **Deterministic** branch: outcome is assembled from **phases of stabilizer rows** paired with destabilizers that have X on `q` (`outcome ^= r_phase[n + r]` pattern).

So **phase bits are load-bearing** for deterministic Z outcomes, not only for aesthetics.

Ancilla hygiene

`reset_z(q)` measures Z and applies `x(q)` if the outcome was 1, so the qubit is returned to (|0\rangle) when used as a **measured ancilla** (used in `BitFlip3Code.measure_syndrome`).

Inspection helpers

- `stabilizer_generators()` — last **n** tableau rows as `(phase_bit, x_row, z_row)`.
- `format_chp_printstate()` — CHP-like `+`/`-` Pauli strings (destabilizers, rule line, stabilizers).
- `format_xz_binary_matrices()` / `format_tableau_debug()` — human-readable dumps for demos and benchmarks.

QEC workflows (`codes.py`)

Code	Encoder	Syndrome	Correction
BitFlip3	`encoder_circuit()` (CNOTs on 3 data qubits)	`read_syndrome(state)` — phase bits from stabilizer rows matching (Z_0Z_1), (Z_1Z_2) (no ancillas); `measure_syndrome(state)` — ancilla CNOT + `measure_z` + `reset_z` on q3/q4	`correct_x_from_syndrome(state, s01, s12)`
Shor9	`encoder_circuit()` (phase spread + H on block roots + intra-block repetition)	`read_syndrome(state)` — all 9 stabilizer phase bits	`correct_x_from_syndrome(state, syndrome)` via `_X_SYNDROME` lookup for single-X errors

Demos use the **ancilla-free read** path where possible (`bitflip3_demo`, `shor9_demo`); tests cover both **measured** syndromes and **Z-parity** behavior (Z errors on the bit-flip code yield trivial syndrome).

Results (tests)

From repo root:

```
python -m pytest -q stabilizer-python/tests
...........                                                              [100%]
11 passed in ~0.02s
```

(Exact timing varies by machine.)

File	What it checks
`test_two_qubit_bell.py`	Bell state (Z_0Z_1) and (X_0X_1) parities via ancilla measurement
`test_bitflip3.py`	X-error syndrome table, Z errors undetected, single-X correction loop
`test_gaussian_elimination.py`	GF(2) RREF, rank, input validation

Novel / distinguishing points (for this codebase)

1. **Explicit mod-4 phase bookkeeping in `_rowmult`**, with a **fully enumerated** `_row_mult_phase`, then **projection to a single sign bit per row** — minimal memory, correct row ops.
2. **Clear split** between immutable-style **`Circuit` recording** (`Op` list, optional `MZ:key` labels) and mutable **`StabilizerState`** simulation.
3. **Pure Python, no NumPy** for the simulator core; GF(2) helpers are separate and reusable.

4. **Two syndrome styles** for the same code: stabilizer-row **phase read** vs **ancilla measure-and-reset**, plus Shor **syndrome lookup** for X correction.
5. **CHP-aligned printing** for cross-checking against CHP / benchmark tooling without leaving Python.

Limits (honest scope)

- Tracks **±** on tableau rows, not arbitrary **(i^k)** phases on every generator in full generality (sufficient for the gates and measurements implemented).
- **Z-basis measurements only** via `measure_z` (appropriate for stabilizer / CHP-style demos).
- Bit-flip and Shor helpers here target **single-qubit X** recovery patterns exercised in demos/tests; full Shor recovery (all Pauli types) is not implemented.

For file references, the phase logic is concentrated in `stabilizer_python/tableau.py` (`_row_mult_phase`, `_rowmult`, `h`, `s`, `x`, `z`, `cnot`, `measure_z`); QEC composition is in `stabilizer_python/codes.py`.